

MonthlyMuse

Presentation script — two speakers

AI-Powered Automated Message Generator for Monthly Posting · ~14 minutes + Q&A · **Speaker A** (blue shirt) opens and handles architecture / engineering · **Speaker B** (green shirt) handles the AI maths and runs the live demo

14 slides

14 minutes

live demo built in

alternating speakers

Q&A prep included

How to use this script

- Text inside a coloured box is **verbatim** — say it as written or in your own words; the meaning must stay.
- **SPEAKER A** and **SPEAKER B** mark who owns the slide.
- **Handoff lines** (in yellow) are spoken by the named speaker to pass the floor — say them out loud so the audience knows who is talking next.
- Timings are cumulative from the start. Keep to them; if you run long, trim the roadmap slide, never the demo.
- Run the demo on `http://localhost:3000` with the seeded account `demo@monthlmuse.app` / `monthlmuse-demo`.

Rehearsal tip: practise the demo transitions once with the site open. The dashboard already shows "November post ready for review (3 candidates)", so the demo starts one click from the review screen.

Slide 1 — Title 0:00 - 0:30

MonthlyMuse — AI-Powered Automated Message Generator for Monthly Posting

Opening

0:30

SPEAKER A

"Good morning everyone. This is MonthlyMuse — a web application that prepares your monthly social post *before you even ask for it*. You define a monthly plan once, and every month the system drafts six candidate messages, ranks them with our own NLP engine, and hands you the best three with a transparent explanation of why they ranked where they did."

SPEAKER B

"Everything you'll see today is built and running — sixty-two automated tests pass, and we'll be demoing the live application, not slides of it."

Handoff — A: "To understand why this is worth building, let's start with the problem."

Slide 2 — The problem 0:30 - 1:30

Small businesses post monthly — and struggle every time

Context

1:30

SPEAKER A

"Think about a small café, a salon, or a tutor. They know they should post once a month. But on the day, they face a blank box: what do I write, does it sound like me, have I already said this last month? The result is either no post, or a generic one that doesn't sound like their brand."

"Existing AI writers give you one shot and no explanation. You get a paragraph, you don't know why it was chosen, and next month the tool has forgotten everything you liked. Nothing compounds."

Handoff — A: "So our goal was specific. B will explain it."

Slide 3 — The goal 1:30 - 2:15

Goal: prepare, don't react — and prove it improves

Objective

2:15

SPEAKER B

"Our goal in one sentence: the system should prepare the monthly post *before* the user asks, show three genuinely different candidates with a visible score breakdown, and then measurably improve from every choice the user makes."

"Three promises. **Automatic** — a scheduler, not a reminder. **Transparent** — you can see every feature that produced the score. And **adaptive** — your selection changes next month's output."

Handoff — B: "Here's what that looks like. A takes you through the flow."

Slide 4 — How it works (user flow) 2:15 - 3:15

The monthly loop

Product flow

3:15

SPEAKER A

"The loop is short. First, you create a **Monthly Plan** — topic, audience, platform, tone, posting day. Then the scheduler detects the upcoming month, pulls the relevant occasion — Diwali, a seasonal moment — and runs the pipeline automatically."

"You get a notification: three ranked candidates are waiting. You open the review screen, pick one, edit one, or reject one with a reason. You approve it, it lands in the calendar, and it's marked published."

"Every one of those actions is recorded as a feedback event. That's where the learning starts."

Handoff — A: "But behind that simple loop sits a fairly serious engine. B."

System architecture — three pieces, one database

Engineering

4:30

SPEAKER B

"The system is three pieces. A **Next.js 14 frontend** that only ever talks to the public API — no secrets in the browser. A **FastAPI backend** with fifty-eight endpoints covering auth, plans, cycles, messages, analytics and the calendar."

"And a **scheduler worker** running on APScheduler, sharing the same code and the same database but started with a different command. The API is stateless, so it can be scaled by just adding instances."

"Underneath is a **sixteen-table schema**: users, preferences, audiences, topics, occasions, plans, cycles, generation requests, messages, feedback events, tone statistics, scheduled posts, metrics, notifications, job runs, and a daily analytics rollup."

Handoff — B: "The interesting part is what runs inside that backend. A."

Fourteen pipeline stages, in one request

Core system

6:00

SPEAKER A

"Generation is a fourteen-stage pipeline. It starts by **understanding context** — plan, audience, occasion, platform — then **retrieves your history**: your past posts, their embeddings, and what you selected or rejected."

"Next it picks **style arms** using Thompson sampling, then makes **one call** to the language model — which over-generates **six** candidates. One call, six candidates: that's deliberate, it's the cost-efficient way to have something to choose from."

"Then the filter chain. **Hard constraints** — length limits per platform and language, banned words, emoji level. **Deduplication** so no two candidates repeat an opening line. Then everything is **embedded** with our own 384-dimensional embedder."

"We compute **six features** per candidate, score them, run **MMR** to make sure the final three are stylistically different, estimate **confidence**, and produce an explanation payload — which is what you actually see on screen."

Handoff — A: "Now the maths. B."

Scoring: $s = F \cdot w$, then MMR, then confidence

Mathematical core

7:30

SPEAKER B

"Each candidate is a row of six features: **relevance, tone, personalization, novelty, length** and **history**. The score is a dot product — **s equals F times w** — with default weights of point three, point two, point two, point one five, point one and point oh five."

"The score alone would give you three near-identical posts. So we add **MMR** — maximal marginal relevance — at lambda zero point seven, which trades relevance against diversity. That's why candidate three is tagged '*exploring a new style*'."

"Confidence comes from a **Dirichlet Monte-Carlo simulation**: two hundred seeded draws, so the same inputs always give the same answer. For rates in the analytics we use **Wilson intervals**, because a naive percentage lies when n is small."

Handoff — B: "Rather than describe the output, let's show it. A drives the demo."

Demo: three ranked candidates with score breakdowns

Live — localhost:3000

9:30

SPEAKER A

"This is the live app. The dashboard says '*November post ready for review — three candidates*'. That message came from the scheduler, not from a button press."

Click → Review candidates.

"Here are the three. Each card shows a **score ring**, a **confidence band**, word and character counts, and how similar it is to your history — twenty percent, thirty-five percent, fifty-three percent. All three are marked *Fresh*, so we're not repeating ourselves."

Click → Score breakdown on candidate 1.

"This is the transparency we promised. Six bars: relevance seventeen, tone thirteen, personalization sixty-one, novelty eighty, length eighty-five, history sixty-seven. You can see exactly which feature earned the top rank."

"And this list on the left is the **why-ranked** explanation in plain English — '*fits your usual voice at sixty-one percent*' and '*twenty percent different from your nearest past post*' — with a link to the exact post it's being compared against."

Handoff — A: "Now watch what happens when we give feedback. B?"

Slide 9 — LIVE DEMO: the learning loop

9:30 - 10:45

Demo: one click changes the next month

Live — feedback → personalization

10:45

SPEAKER B

"I'll like this first style. Watch the network — the feedback call returns two hundred OK, and the request refetches."

Click → 'Like this style', then point at the Recent activity feed on the dashboard.

"That single click updated four things at once. It moved the **preference vector** with an exponential moving average. It updated the **Beta statistics** for that style arm, so Thompson sampling will favour it next time. It nudged the **weight vector** with a constrained gradient step. And it updated your **tone counts**."

"Reject does the same thing with a reason attached — if you reject for '*too similar*', the novelty weight goes up. That's the difference between a generator and a system that learns."

Handoff — B: "A shows where that shows up over time."

Slide 10 — Analytics

10:45 - 11:45

Analytics: proving the improvement

Evaluation

11:45

SPEAKER A

"Three dashboards. **Usage** — how many messages generated, time to approve. **Content** — tone mix, topic distribution, novelty trend across months, and a repetition alert that fires when two posts are more than eighty-five percent similar."

"And **Quality** — selection rate with a Wilson confidence interval, top-one agreement, mean reciprocal rank, and the correlation between score and what you actually chose."

"Right now: nine messages generated, twenty-two percent selection rate with a ninety-five percent interval of six to fifty-five percent. The interval is wide because n is nine — which is exactly why we show it."

Handoff — A: "B takes the technology summary."

Slide 11 — Technology stack 11:45 – 12:45

Technology stack

Engineering choices

12:45

SPEAKER B

"**FastAPI** on Python 3.10 with Pydantic for validation, **SQLAlchemy 2** over SQLite by default — swap one environment variable and it's PostgreSQL."

"The AI engine is **pure NumPy and SciPy** — no GPU, no heavy framework. Our own 384-dimensional embedding, our own preprocessing, our own logistic-regression tone classifier trained on eight hundred labelled rows."

"Generation sits behind an **adapter** — template, OpenAI, Anthropic or Ollama — with retry, a circuit breaker and cost metering. The template engine is the default, so the demo works offline with zero API cost."

"On the front, **Next.js 14** with TypeScript, TanStack Query, React Hook Form with Zod, and Recharts for the visualisations."

Handoff — B: "A on quality."

Slide 12 — Quality & testing 12:45 – 13:30

Quality: 62 tests, including every golden value

Verification

13:30

SPEAKER A

"Sixty-two tests, all passing. Thirty-five of them are **golden-value tests** — we wrote down the expected number first, then made the code produce it. The gradient step lands on exactly 0.161, the Wilson interval for three successes in ten trials is 0.108 to 0.603, and the MMR picks candidates A, C and F."

"Beyond that: twelve pipeline tests for the constraints, a full API journey test, and four scheduler automation tests. Type-checking and the production build are clean, and the whole thing runs in CI on Python 3.10 and 3.11."

Handoff — A: "B finishes."

Where it stands, and what's next

Closing

14:00

SPEAKER B

"The MVP is **complete** — every row of the scope is built, tested and running. The code is on GitHub, there's Docker Compose for the full stack, a CI pipeline, and a nine-page technology document."

"Next: deploying it live on Render with managed Postgres, connecting a hosted language model so the copy is AI-written rather than template-written, and adding a browser end-to-end test."

SPEAKER A

"To leave you with the one idea: this isn't a text box with an API behind it. It's a system that prepares, explains itself, and gets measurably better every month you use it."

"Thank you — we're happy to take questions."

Slide 14 — Q&A preparation

Anticipated questions, with the suggested answer and who should take it.

Question	Who	Suggested answer
Why is the generated text a bit awkward?	A	"The demo runs on the offline template engine so it costs nothing and works without keys. The OpenAI/Anthropic/Ollama adapters are built and tested — we add a key and the same six candidates come from a real model. Everything downstream — scoring, ranking, personalization — is identical."
Why not just use ChatGPT?	B	"A single model call gives you one answer with no ranking and no memory. We over-generate six, then filter, embed, score and diversify with our own engine — and every user action feeds back into next month's ranking. The model is one component, not the product."
How is this different from a prompt?	B	"A prompt has no state. Ours has a preference vector, Beta statistics per style arm, learned length stats and a weight vector that updates with a constrained gradient step — all visible in Settings."
Is it really automatic?	A	"Yes — APScheduler runs a daily scan, creates the cycle for the upcoming month, gathers the occasion context, generates, and notifies at T-3 and T-1 days. Preparation is idempotent, so a restart can't duplicate anything."
Why SQLite and not PostgreSQL?	A	"The target machine has no Docker or Postgres. The schema is portable — string enums, UTC-normalised timestamps, vectors as float32 blobs — so setting <code>MM_DATABASE_URL</code> to a managed Postgres is the only change. That's the deployment plan."
Isn't 22% selection rate low?	B	"n is nine, so the Wilson interval is six to fifty-five percent — too early to read as a quality signal. It's also counting 'left unselected' events. The metric exists precisely so we don't over-claim from small numbers."
How do you stop it repeating itself?	B	"Three ways: n-gram Jaccard deduplication at generation, cosine similarity against your full history with a novelty feature in the score, and MMR so the three shown are stylistically distinct. There's also a repetition alert in analytics at cosine 0.85."
What about privacy / API keys in the browser?	A	"The frontend only calls the public API and holds no secrets. Refresh tokens are httpOnly cookies; passwords are Argon2id. Secrets live in platform environment variables, never in the repository."
What's the hardest part you'd redo?	Either	"Getting the golden tests written first. Several real bugs — candidates never being embedded, a naive/aware datetime mix, MMR returning two candidates from the same style arm — were caught by those tests before they could reach the UI."
Can it handle multiple users?	A	"Yes — every query is scoped to the authenticated user, there's per-user rate limiting, and the API is stateless so instances can be added behind a load balancer. The scheduler worker is separate and shares only the database."

Timing summary

Slide	Topic	Speaker	Cumulative
1	Title	A then B	0:30
2	The problem	A	1:30
3	The goal	B	2:15
4	User flow	A	3:15
5	Architecture	B	4:30
6	AI pipeline (14 stages)	A	6:00
7	The maths	B	7:30
8	Demo — review screen	A	9:30
9	Demo — learning loop	B	10:45
10	Analytics	A	11:45
11	Tech stack	B	12:45
12	Quality & tests	A	13:30
13	Live & next steps	B then A	14:00
14	Q&A	both	—